

Server-Side Development with Node.js

Node.js Runtime · Event-Driven Architecture · Building Basic Servers with Core Modules

MODULE	Server-Side Development — MERN Stack
SESSION FOCUS	Node.js runtime, event loop, EventEmitter, core HTTP server
NEXT SESSION	Express.js
SUGGESTED DURATION	2.5 – 3 hours (Theory ~60 min, Practicals ~90-120 min)
PREREQUISITES	Basic JavaScript (functions, callbacks), terminal familiarity

Prepared for classroom delivery — includes guided practicals and discussion prompts

Session Objectives

- By the end of this session, students will be able to:
- 1. Explain what Node.js is and how it differs from browser JavaScript
 - 2. Describe Node.js's single-threaded, event-driven, non-blocking I/O model
 - 3. Use the built-in `EventEmitter` to create and handle custom events
 - 4. Build a working HTTP server using only Node.js core modules (no frameworks)
 - 5. Understand *why* Express.js will matter next session (the pain points core `http` leaves unsolved)

Part 1 — What is Node.js? (Theory — 20 min)

1.1 Definition

Node.js is **not a language** and **not a framework** — it's a **JavaScript runtime environment** built on Google Chrome's **V8 engine**, which allows JavaScript to run outside the browser (on servers, desktops, IoT devices, etc.).

1.2 Why Node.js for the Server?

Before Node.js, JavaScript only ran inside browsers. Node.js took the V8 engine and added:

- File system access (`fs`)
- Networking (`http` , `net`)
- Process/OS access (`process` , `os`)
- A module system (`require` / `import`)

This let developers use **one language (JavaScript) for both frontend and backend** — the foundation of the MERN stack (MongoDB, Express, React, Node).

1.3 Key Characteristics

Feature	Explanation
Single-threaded	Node runs your JS code on ONE main thread (not multi-threaded like Java/traditional servers)
Non-blocking I/O	Slow operations (file reads, DB queries, network calls) don't freeze the thread — they run in the background
Event-driven	Node reacts to <i>events</i> (a request arriving, a file finishing reading, a timer firing) rather than executing top-to-bottom and waiting
Asynchronous	Uses callbacks, Promises, and <code>async/await</code> to handle results whenever they're ready

1.4 Analogy to Use in Class

CLASSROOM ANALOGY

A traditional (blocking) server is like **one waiter** who takes an order, walks it to the kitchen, and **stands there waiting** until the food is cooked before serving the next customer.

Node.js is like **one waiter** who takes an order, hands it to the kitchen, and *immediately* goes to take the next customer's order. When the kitchen (background system) finishes a dish, it rings a bell (**event**), and the waiter delivers it. One waiter, but nobody sits around blocked.

Part 2 — Event-Driven Architecture (Theory — 25 min)

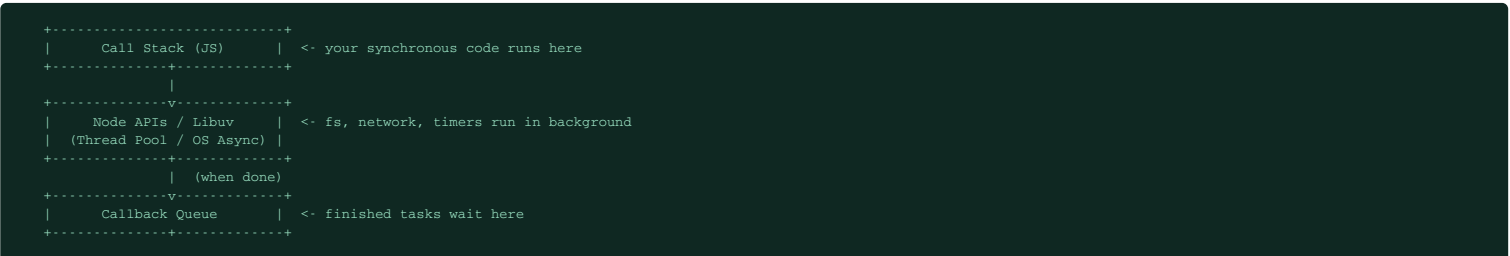
2.1 The Event Loop

Node.js uses a mechanism called the **Event Loop** to handle many operations concurrently on a single thread.

Simplified flow:

1. Your script runs (synchronous code executes first, top to bottom)
2. Async operations (timers, file reads, network requests) are handed off to the system/background
3. Node keeps running other code — it does **not** wait
4. When a background operation finishes, its **callback** is placed in a queue
5. The **Event Loop** continuously checks: "Is the main thread free? Is there a callback waiting?" — if yes, it runs it

Draw this on the board (or use the diagram practical below):



2.2 The EventEmitter — Node's Core Building Block

Almost everything in Node (HTTP servers, streams, file watchers) is built on top of the `EventEmitter` class from the `events` core module. It follows the **publish/subscribe pattern**:

- You **emit** (fire) a named event
- One or more **listeners** react to it

This is the same pattern used later by `req / res` objects in Express, and by React's event handling — so understanding it now pays off repeatedly.

Part 3 — Core Modules for Building Servers (Theory — 15 min)

Node ships with built-in modules — no `npm install` needed:

Module	Purpose
http	Create web servers, handle requests/responses
fs	Read/write files
path	Handle and normalize file paths safely
os	Get system info (CPU, memory, platform)
url	Parse URL strings and query parameters
events	Custom event emitters

EMPHASIZE TO STUDENTS

Express.js (next session) is built on top of `http` — it doesn't replace it, it wraps it to remove boilerplate (routing, middleware, body parsing, etc.). Today's session is what Express is hiding from you.

Part 4 — Practicals

Set up: Node.js installed (LTS version), a code editor (VS Code), terminal access.

5 min

Create a file `hello.js` :

Run it:

15 min

Create `async-demo.js` :

Run it and discuss the output order — this is the "aha" moment for event-driven behavior. The script doesn't pause at `setTimeout` ; it registers the callback and moves on.

Extension exercise: Add a `fs.readFile` call reading a large file alongside a `console.log` after it, and ask students to predict the output order before running.

Create `events-demo.js` :

```
const EventEmitter = require('events');

class OrderSystem extends EventEmitter {}
const orderSystem = new OrderSystem();

// Listener (subscriber)
orderSystem.on('orderPlaced', (orderName, price) => {
  console.log(`New order received: ${orderName} - Rs. ${price}`);
});

orderSystem.on('orderPlaced', (orderName) => {
  console.log(`Kitchen notified to prepare: ${orderName}`);
});

// Emit (publish) the event
orderSystem.emit('orderPlaced', 'Chicken Kottu', 850);
orderSystem.emit('orderPlaced', 'Fried Rice', 650);
```

DISCUSSION POINT

One event, multiple independent listeners react. This is exactly the pattern behind `server.on('request', ...)` used in the next practical.

Practical 3 — Build a Basic HTTP Server

30 min

Create `basic-server.js` :

```
const http = require('http');

const server = http.createServer((req, res) => {
  console.log(`Incoming request: ${req.method} ${req.url}`);

  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from my Node.js server!');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Run it, open `http://localhost:3000` in a browser.

Then extend it — manual routing (no framework):

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Welcome to the Home Page');
  } else if (req.url === '/about' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('This is the About Page');
  } else if (req.url === '/api/students' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify([
      { id: 1, name: 'Kavindu' },
      { id: 2, name: 'Thanuja' }
    ]));
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 - Page Not Found');
  }
});

server.listen(3000, () => console.log('Server running on port 3000'));
```

CLASS DISCUSSION — BRIDGE TO NEXT SESSION

Ask students:

- "What happens when we need 20 routes? Is this `if/else` chain scalable?"
- "How would you handle POST data here?"
- "What if two routes need the same logic (e.g., checking a login token) — do we repeat code everywhere?"

→ These pain points are exactly what Express.js solves. Write their answers on the board and revisit them at the start of next session.

Create a folder `public` with an `index.html` file inside. Then `file-server.js` :

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    const filePath = path.join(__dirname, 'public', 'index.html');

    fs.readFile(filePath, 'utf8', (err, data) => {
      if (err) {
        res.writeHead(500);
        res.end('Server Error');
        return;
      }
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end(data);
    });
  } else {
    res.writeHead(404);
    res.end('Not Found');
  }
});

server.listen(3000, () => console.log('File server running on port 3000'));
```

KEY TEACHING POINT

`fs.readFile` is asynchronous — the server keeps handling other requests while the file is being read from disk. Contrast with `fs.readFileSync` (blocking) and briefly demonstrate why the sync version is discouraged inside a server handler.

Give students this task to complete individually or in pairs:

Build a Node.js core-module-only server (`student-api.js`) that:

- 1. Serves a JSON array of at least 5 objects at `GET /api/students`
- 2. Returns a single student by ID at `GET /api/students/:id` — *hint*: they'll need to manually parse `req.url` using the `url` module, since there's no automatic route-parameter parsing without a framework
- 3. Returns a proper 404 JSON response (not plain text) for unknown routes
- 4. Logs every incoming request's method, URL, and timestamp to the console using an EventEmitter-based logger (bonus)

WHY THIS MATTERS

This exercise deliberately makes students feel the friction of manual routing and parsing — set up the "why Express?" hook for next class.

Wrap-Up Discussion (10 min)

Ask the class:

- 1. Why is Node.js good for I/O-heavy applications (APIs, chat apps, streaming) but not ideal for CPU-heavy tasks (video encoding, heavy computation)?
- 2. What did we have to do manually today that felt repetitive or error-prone?
- 3. Preview: "Next session, we'll install Express.js and rebuild today's server in about 15 lines instead of 60 — and you'll understand exactly what it's doing under the hood, because you built it yourself first."

Quick Reference — Commands Used Today

```
node -v          # check Node version
node filename.js # run a Node script
npm -v           # check npm version
```

Suggested Assessment Questions (for next class recap quiz)

- 1. Explain in your own words why `setTimeout(fn, 0)` still runs after synchronous code.
- 2. What module would you use to create a custom event emitter?
- 3. What does `res.writeHead()` do, and why must it come before `res.end()` ?
- 4. Name two limitations of using raw `http` for routing that Express solves.